

Как общий контекст в RAG помогает разработке AC по SDD с 300+ микросервисами

От короткого требования — к проверяемому плану изменений, связанному с документами, кодом и зависимостями системы.

проблема решение результат

«Перенести дату платежа по кредиту физлица».

Сначала надо понять, **что уже есть**.

Задача

что должно измениться для пользователя

Кто отвечает

где начинается процесс и кто его меняет

Что уже работает

правила · код · спецификации · ADR

Кого заденет

API · Kafka · данные · другие команды

Что делать

сервисы · тесты · порядок изменений

Код писать пока рано: сначала надо подтвердить текущее поведение, владельцев, контракты и границы изменения.

ПОЧЕМУ ЭТО БОЛЬШАЯ СИСТЕМА

300+ сервисов — это не 300 папок.
Это сотни границ.

300+

сервисов, репозиториев
и контекстов команд

У сервисов разные владельцы и релизы

Одна команда не может за другую принять решение, изменить контракт или выпустить сервис.

Границы проходят не только по коду

HTTP, Kafka, данные, права доступа и правила бизнеса связывают репозитории между собой.

ПОЧЕМУ НЕ РЕШИТЬ ЭТО ОДНИМ МЕСТОМ

Сложить всё в один репозиторий — значит создать ещё одну очередь.

Центральная копия

Кто-то должен её поддерживать
изменения ждут общего владельца

Копия быстро расходится с кодом
команда уже поменяла сервис, а сводка ещё нет

Поиск всё равно остаётся ручным
нужно понять, какой кусок общего знания относится к задаче



Единый доступ

Истина остаётся у команды
код и спецификация меняются там, где живёт сервис

Система строит маршрут
сначала находит затронутые сервисы, потом детали

Человек видит основания
в ответе остаются ссылки на источник и неизвестные

С ЧЕГО НАЧАЛИ

Сначала посмотрели, как ответ ищут на самом деле.

Где лежит правда

Спецификации, ADR, контракты, код и конфигурация уже существуют — но в разных местах.

Как идёт поиск

Сначала знают один сервис или одного человека; дальше идут по ссылкам, чатам, коду и соседним командам.

Где всё тормозит

На каждом шаге нужно угадывать: этот ли репозиторий, свежий ли документ и все ли соседи уже найдены.

Знания есть. Нет быстрого пути от задачи к нужным знаниям.

ЧТО ДОБАВЛЯЕМ В СИСТЕМУ

Не переносим знания в центр. Строим единый путь к источникам.

Оставляем у команд

Код и спецификация рядом с сервисом
владелец меняет то, за что отвечает

Свой ритм поставки
команды не ждут общего редактора

Своя ответственность
актуальность остаётся там, где принимают решение



Добавляем RAG и граф

Единая точка входа
вопрос задаётся по смыслу задачи

Маршрут по системе
граф находит вероятных соседей и границы

Контекст по делу
RAG поднимает источники только по нужным сервисам

RAG собирается из источников, которые команды **уже ведут**.

Документы и Git

SSOT · OpenSpec · ADR · код

Нормализация

заголовки · таблицы · блоки
кода

Фрагменты с паспортом

сервис · источник · revision

Индексы

по сервисам и доменам

Поиск с evidence

короткий ответ + ссылка на
первоисточник

В индекс не попадает «мнение модели»: у найденного фрагмента остаются сервис, путь или URL источника и revision, по которым его можно проверить.

Хеши в RAG нужны в двух разных местах.

Контроль изменений

- У документа есть content hash.
- У чанка — стабильный ID из документа, раздела и текста.
- При переиндексации векторы неизменившихся чанков берутся из cache.

Быстрый поиск по терминам

- Слова и символьные триграммы превращаются в признаки.
- SHA-256 кладёт каждый признак в ячейку вектора со знаком.
- Получается локальный нормализованный lexical vector.

Это разные задачи: один хеш экономит повторную работу при обновлении, другой создаёт вектор для автономного поиска по совпадающей терминологии.

Видеокарта ускоряет semantic-поиск, но не является обязательной.

Hash provider

Режим по умолчанию: слова и триграммы
→ настраиваемый вектор. Нет модели,
GPU и сетевого вызова.

Local semantic model

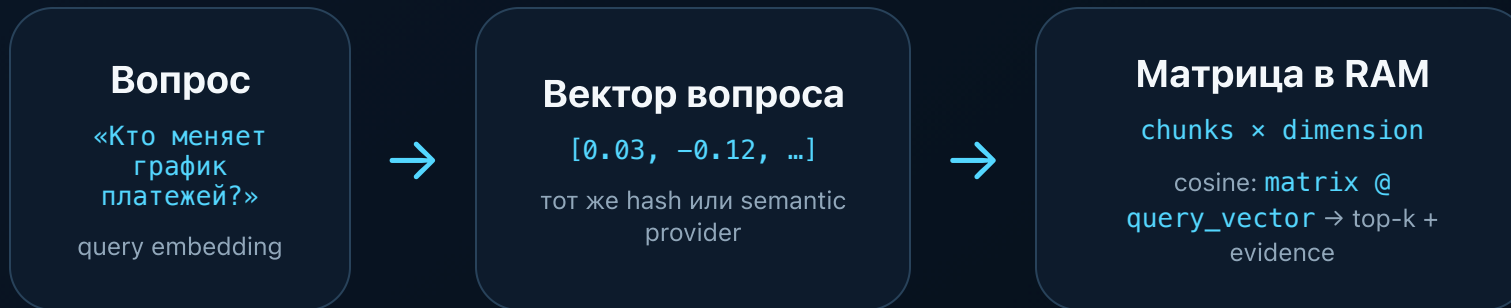
Локальная Sentence Transformers-модель
даёт поиск по смыслу и синонимам; веса
модели заранее лежат на машине.

Выбор устройства

Авто-режим выбирает CUDA, затем MPS,
затем CPU. На CPU работает медленнее,
но работает.

Данные не уходят во внешний embedding API: при `local_files_only` недостающая модель даёт ошибку, а не скачивается из сети.

Вопрос и документ переводятся одним **provider**'ом. Дальше — обычная математика.



Здесь нет отдельной vector DB: индекс хранит NumPy-матрицу, считает полный cosine scan в памяти и возвращает короткие выдержки со ссылками на источники.

Граф строится из связей в системе, а не из догадок модели.

Исходники и контракты

клиенты HTTP · OpenAPI · Kafka ·
конфигурация

Извлекаем связи

кто вызывает, публикует, читает
и владеет данными

Граф сервисов

узлы · рёбра · направление ·
confidence

Evidence и revision

видно, из какого источника взята
каждая связь

Это навигация, а не runtime trace: граф помогает выбрать маршрут анализа, но важный вывод всё равно проверяется по коду, контракту и документу.

Базовый граф читается из репозиториев. Сервисы запускать не нужно.

Находим границы

Maven/Gradle-модули, `spring.application.name`, `manifest` и конфигурация подсказывают, где начинается сервис.

Читаем исходники

Tree-sitter разбирает Java и Kotlin: контроллеры, клиенты, `listeners`, `producers` и точки входа.

Публикуем snapshot

Узлы, связи, `evidence` и `revision` сохраняются в `versioned JSON` для повторяемого анализа.

Для быстрого графа не нужно запускать сервисы, сборку или вызывать API системы: достаточно исходников и конфигурации из Git.

Ребро появляется не «по похожему имени».

Его нужно сопоставить с контрактом.

Находим источник

Feign · WebClient · RestTemplate ·
KafkaTemplate · listener

Нормализуем значения

разрешаем properties и простой
YAML, приводим path к шаблону

Сверяем контракт

HTTP: alias/host + method/path
Kafka: точный topic

Сохраняем evidence

repository · commit · file:line ·
snippet · extractor

Например, совпадение URL ещё не достаточно: для HTTP нужны сервис-назначение и нормализованные method + path; для Kafka — один и тот же topic.

Граф не скрывает сомнения. **Неизвестное** тоже становится результатом.

DECLARED / HIGH

Есть декларация или сильное source evidence: связь можно использовать как маршрут анализа.

MEDIUM / LOW

Есть кандидат: похожий клиент, интерфейс или неполный контракт. Его нужно проверить.

UNRESOLVED

Динамический URL, topic или несколько целей. Граф не выбирает ответ за человека.

Опциональная проверка GigaCode не рисует граф заново: она разбирает ограниченные пакеты кандидатов и принимает связь только с реальным source evidence и точной целью.

План не появляется из промпта. Он собирается **по шагам**.

01

Разобрать задачу

цель, актор, ожидаемый результат и ограничения

02

Найти точку входа

корневой сервис или список кандидатов

03

Пройти по графу

выбрать соседей и границы репозиториев

04

Поднять источники

найти контракты, правила и код в нужных индексах

05

Подтвердить score

отделить затронутые сервисы от кандидатов

06

Собрать план

изменения, контракты, rollout, тесты и риски

07

Проверить план

найти выдуманные факты, конфликты и пробелы

08

Подготовить Jira

показать preview и ждать явного подтверждения

На каждом шаге можно остановиться: если не найдена точка входа, нет источника или сервис остаётся кандидатом, это фиксируется как неизвестное, а не маскируется красивым планом.

Планировщик выдаёт не сочинение, а строгий **PLAN_OUTPUT_V1**.

`$generate-cross-service-
feature`

Формулирует задачу

цель, акторы, сценарии,
ограничения и критерии
приёмки

`$verify-cross-service-feature`

Проверяет исходные факты

не даёт выдать требования или
догадки за текущее состояние

`$plan-feature-with-system-
graph`

Собирает единый план

бизнес-резюме · сервисы ·
изменения · тесты · риски · Jira
drafts

`$verify-cross-service-feature`

Проверяет готовый план

сверяет сервисы, контракты,
evidence и оставшиеся
неизвестные

Бизнес читает резюме и риски; инженер — изменения, контракты, rollout и тесты. Оба смотрят на один и тот же план.

На экране — план, который можно обсуждать и проверять.

● ● ● GigaCode · feature planning

План кросс-сервисной фичи

Сервисы · evidence · изменения · тесты · неизвестные

Маршрут

корневой и затронутые сервисы

Evidence

ссылки на документы, код и связи графа

Изменения

контракты, данные, тесты и rollout

Unknowns

что ещё нельзя считать подтверждённым

После подтверждения план превращается в Jira-задачи по сервисам

● ● ● GigaCode · Jira publication

Предпросмотр → выбор проекта → Task

Черновики по сервисам · явное подтверждение · созданные ключи задач

Сначала preview

до выбора проекта Jira не изменяется

Явное подтверждение

пользователь выбирает один доступный проект

Проверка результата

созданная задача читается обратно

1 проект

один выбранный пользователем Jira-проект

1 сервис = 1 Task

только для подтверждённых сервисов

0 скрытых записей

без подтверждения публикации нет

У важного вывода есть источник. Или честная пометка: «не знаем».

Факт можно открыть

Документ, код или связь графа остаются рядом с выводом. Предлагаемое изменение не выдаётся за уже работающую функцию.

План читает второй skill

Он ищет несуществующие сервисы, перепутанные контракты и противоречия между источниками.

Пробел не прячется

Устаревший индекс, слабая связь графа или отсутствие источника становятся вопросом к команде и задачей проверки.

Если нет источника — это не факт в плане, а открытый вопрос.

Общий контекст делает SDD рабочим на масштабе

Для бизнеса

- затронутые команды и сервисы видны до оценки;
- риски и неизвестные появляются до обещаний;
- план и задачи связаны с исходным требованием.

Для разработки

- поиск начинается с вопроса, а не обхода репозитория;
- ключевые выводы можно проверить по evidence;
- план учитывает контракты, данные, тесты и rollout.

Главный результат: единый RAG не заменяет владельцев знаний — он связывает их источники в повторяемый и проверяемый процесс разработки.